

Term Project: Linux System(Socket programming)

과목: 리눅스 시스템(류은석 교수님)

2019312014 컴퓨터교육과 박병준

2020 년 6 월 17 일 최종 제출본

(Mission1 은 6 월 6 일 제출본과 동일한 내용입니다. 이번에 Mission2 의 내용을 추가했습니다.)

Mission1. TCP 를 이용한 서버와 클라이언트를 구현하시오.

C/C++를 사용하되, 리눅스에서 빌드가능하고 동작가능해야 하며, 다음과 같이 동작한다.

(1) Client 가 알려진 서버 주소로 접속하여 Hello Server 메시지 전달, (2) 서버는 받아서 이를 화면에 printf 하고 클라이언트에게 역시 Hello Client 메시지 전달, (3) Client 는 이를 받아서 화면에 출력.

1. 소스코드

1-1. server 소스코드

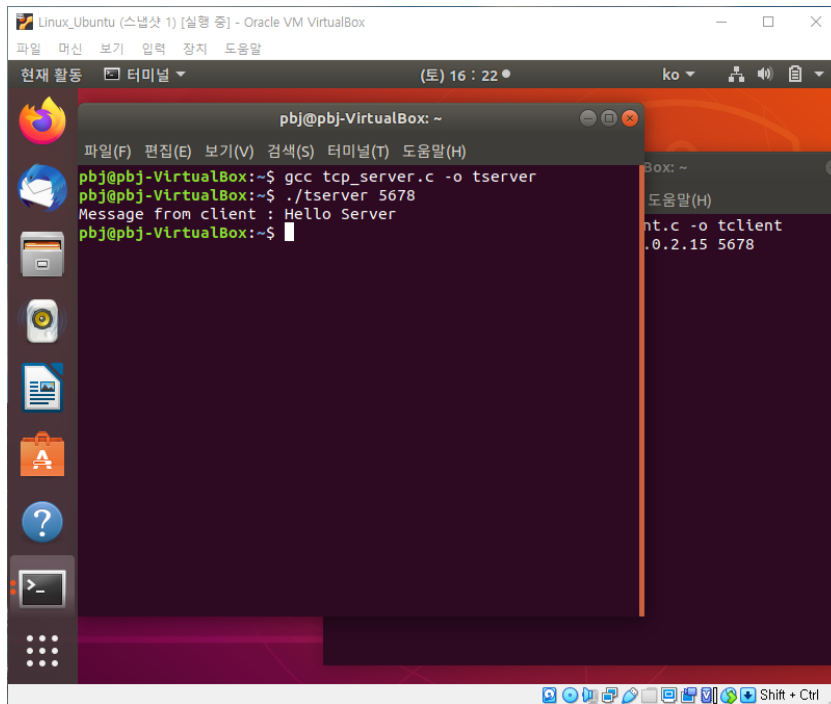
```
1 #include <stdio.h>
2 #include <stdlib.h> // atoi
3 #include <string.h> // memset
4 #include <unistd.h> // sockaddr_in, read(), write()
5 #include <arpa/inet.h> // htonl, htons, INADDR_ANY
6 #include <sys/socket.h>
7
8 void error_handling(char* message);
9
10 int main(int argc, char* argv[])
11 {
12     int serv_sock;
13     int clnt_sock;
14
15     //sockaddr_in은 소켓 주소의 틀을 형성해주는 구조체, IPv4용 구조체
16     struct sockaddr_in serv_addr;
17     struct sockaddr_in clnt_addr; // accept()에서 사용
18     socklen_t clnt_addr_size;
19
20     char client_message[30]; // client가 server에 보낸 메시지를 저장
21     char server_message[] = "Hello Client"; // server가 client에 보내는 메시지
22
23     if (argc != 2) { // 입력 시 port번호가 누락된 경우
24         printf("Error : %s <port>\n", argv[0]);
25         exit(1);
26     }
27
28     //소켓 생성
29     //socket함수는 int형을 반환하는 socket(int domain, int type, int protocol)형태임
30     serv_sock = socket(PF_INET, SOCK_STREAM, 0); // PF_INET은 IP를 사용하는 프로토콜 체계, SOCK_STREAM은 TCP방식을 의미, 0은 type에서 미리 정해진 프로토콜 방식을 따름
31     if (serv_sock == -1)
32         error_handling("socket error");
33
34     //serv_addr의 주소를 초기화한 후, IP주소와 포트 지정
35     memset(&serv_addr, 0, sizeof(serv_addr));
36     serv_addr.sin_family = AF_INET; // 타입: IPv4(IP를 사용하는 주소체계)
37     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY); // htonl(long형 데이터의 바이트 오더를 호스트(리틀엔디안)방식에서 네트워크(빅엔디안)방식으로 변환)로 IP주소 저장
38     serv_addr.sin_port = htons(atoi(argv[1])); // port
39
40     //IP주소와 포트를 소켓에 결합
41     if (bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
42         error_handling("bind error");
43
44     //연결을 기다림(대기열은 5개)
45     if (listen(serv_sock, 5) == -1)
46         error_handling("listen error");
47
48     //연결 요청을 수락하는 단계
49
50     //연결 요청을 수락하는 단계
51     clnt_addr_size = sizeof(clnt_addr);
52     clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_addr, &clnt_addr_size);
53     if (clnt_sock == -1)
54         error_handling("accept error");
55
56     //데이터 주고받기
57
58     //1. client가 보낸 메시지 받기
59     if (read(clnt_sock, client_message, sizeof(client_message) - 1) == -1)
60         error_handling("read error");
61     printf("Message from client : %s\n", client_message);
62
63     //2. server가 "Hello Client"메시지 보내기
64     write(clnt_sock, server_message, sizeof(server_message));
65
66     //소켓 닫기
67     close(clnt_sock);
68     close(serv_sock);
69     return 0;
70
71 void error_handling(char* message)
72 {
73     fputs(message, stderr);
74     fputc('\n', stderr);
75     exit(1);
76 }
```

1-2. client 소스코드

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6  #include <sys/socket.h>
7
8  void error_handling(char* message);
9
10 int main(int argc, char* argv[])
11 {
12     int clnt_sock;
13
14     struct sockaddr_in serv_addr;
15     char client_message[30] = "Hello Server"; // client가 server에 보내는 메시지
16     char server_message[30]; // server가 client에 보낸 메시지를 저장
17
18     if (argc != 3) { // 입력 시 IP주소나 port번호가 누락된 경우
19         printf("Error : %s <IP> <port>\n", argv[0]);
20         exit(1);
21     }
22
23     //TCP통신, IPv4 도메인을 위한 소켓 생성
24     clnt_sock = socket(PF_INET, SOCK_STREAM, 0);
25     if (clnt_sock == -1)
26         error_handling("socket error");
27
28     //서버의 주소체계, IP, port 저장
29     memset(&serv_addr, 0, sizeof(serv_addr));
30     serv_addr.sin_family = AF_INET;
31     serv_addr.sin_addr.s_addr = inet_addr(argv[1]);
32     serv_addr.sin_port = htons(atoi(argv[2]));
33
34     //client소켓에 server를 연결하는 connect과정
35     if (connect(clnt_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
36         error_handling("connect error!");
37
38     //데이터 주고받기
39
40     //1. client가 "Hello Server"메시지 보내기
41     write(clnt_sock, client_message, sizeof(client_message));
42
43     //2. server가 보낸 메시지 받기
44     if (read(clnt_sock, server_message, sizeof(server_message) - 1) == -1)
45         error_handling("read error!");
46     printf("Message from server: %s\n", server_message);
47
48     close(clnt_sock);
49     return 0;
50 }
51
52 void error_handling(char* message)
53 {
54     fputs(message, stderr);
55     fputc('\n', stderr);
56     exit(1);
57 }
58
59
```

2. 실행 화면

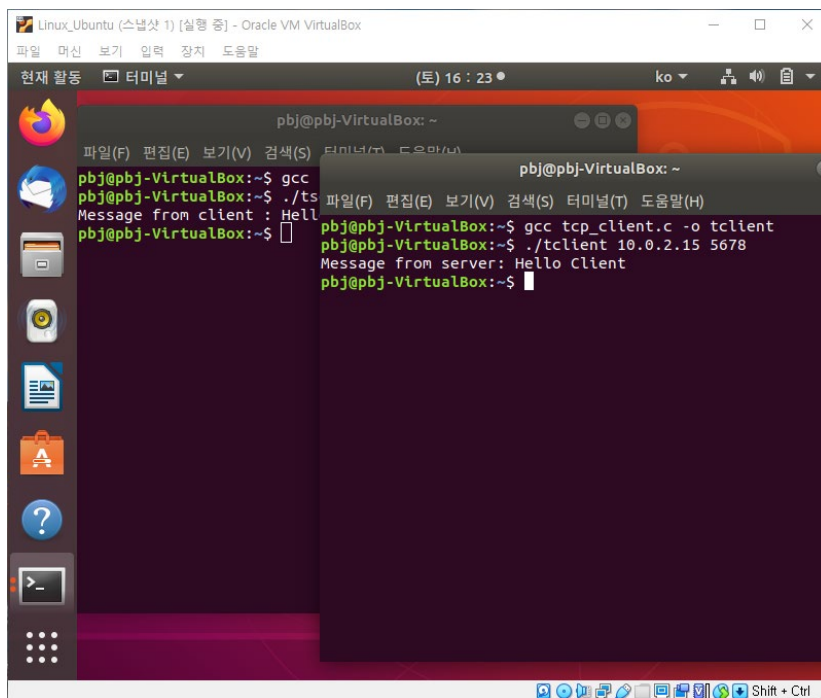
2-1. server 쪽 실행 화면



The screenshot shows a Linux terminal window titled "Linux_Ubuntu (스냅샷 1) [실행 중] - Oracle VM VirtualBox". The terminal output is as follows:

```
pbj@pbj-VirtualBox: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
pbj@pbj-VirtualBox:~$ gcc tcp_server.c -o tserver  
pbj@pbj-VirtualBox:~$ ./tserver 5678  
Message from client : Hello Server  
pbj@pbj-VirtualBox:~$
```

2-2. client 쪽 실행 화면



The screenshot shows a Linux terminal window titled "Linux_Ubuntu (스냅샷 1) [실행 중] - Oracle VM VirtualBox". The terminal output is as follows:

```
pbj@pbj-VirtualBox: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
pbj@pbj-VirtualBox:~$ gcc tcp_client.c -o tclient  
pbj@pbj-VirtualBox:~$ ./tclient 10.0.2.15 5678  
Message from server: Hello Client  
pbj@pbj-VirtualBox:~$
```

3. 구현/개발 내용

<https://blog.naver.com/kim2733137/221936874713> 이곳의 소스코드를 참고하여 Mission1 을 구현해보았습니다. 다만 참고한 소스코드에 대해서는 코드별로 이 코드가 왜 쓰였는지 주석을 달았고, 코드의 흐름을 깨지 않는 선에서 예러 메시지를 추가하거나 변형하는 등 여러 시행착오를 거치면서 코드 내용을 최대한 습득했습니다. 구체적으로 ./tserver 를 입력하여 server 를 실행할 때 ./tserver 입력 후 port 번호를 입력하지 않으면 port 번호를 입력하라는 예러 메시지 기능을 추가했습니다. 마찬가지로 ./tclient 를 입력하여 client 를 실행할 때도 ./client 입력 후 IP 주소나 port 번호를 입력하지 않으면 IP 주소와 port 번호를 입력하라는 예러 메시지 기능을 추가했습니다.

다른 코드는 대부분 참고한 소스코드와 유사하지만, 메시지를 주고받는 과정에서의 코드는 직접 구현했습니다. Server 쪽에서는 client 가 보낸 메시지를 받는 기능과 "Hello Client" 메시지를 보내는 기능을 구현했고, Client 쪽에서는 "Hello server" 메시지를 보내는 기능과 server 가 보낸 메시지를 받는 기능을 구현했습니다.

Mission 2. 위의 결과물을 변형하여 지정된 영화 파일을 서버에서 클라이언트로 전송하도록 구현하시오. 다음과 같이 동작한다. (1) 클라이언트는 서버에게 접속하여 Hello Server 메시지를 전송, (2) 서버는 미리 지정된 영화 파일 (예로 1GB 의 test.mp4 파일)을 접속한 Client 로 발송하되, 1460 바이트의 패킷으로 나누어 계속 전달, (3) 클라이언트는 서버로부터 받아들이는 패킷들을 메모리 버퍼 또는 파일버퍼에 모았다가 최종 file write 를 하여 저장함. (4) 그 후, debug 를 위해 원본 파일과 받은 파일의 내용 및 사이즈가 같은지 비교 (예: cmp 나 diff 명령어 사용).

1. 소스코드

1-1. server 소스코드

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7
8 #define BUFSIZE 1460
9
10 void error_handling(char* message);
11
12 int main(int argc, char** argv)
13 {
14     int serv_sock;
15     int clnt_sock;
16     FILE* fp; // 파일 포인터(.mp4파일을 가리킬)
17
18     struct sockaddr_in serv_addr;
19     struct sockaddr_in clnt_addr;
20     socklen_t clnt_addr_size;
21
22     char buf[BUFSIZE];
23     char cbuf[BUFSIZE];
24     char client_message[30];
25     char client_debug[30];
26     char server_message[] = "Good Bye Client";
27
28     int read_cnt;
29     int filesize;
30
31     if (argc != 2) {
32         printf("Error : %s <port>\n", argv[0]);
33         exit(1);
34     }
35
36     //소켓 생성
37     serv_sock = socket(PF_INET, SOCK_STREAM, 0);
38     if (serv_sock == -1)
39         error_handling("socket error");
40
41     //serv_addr 주소 지정
42     memset(&serv_addr, 0, sizeof(serv_addr));
43     serv_addr.sin_family = AF_INET;
44     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
45     serv_addr.sin_port = htons(atoi(argv[1]));
46
47     //bind
48     if (bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
49         error_handling("bind error");
50 }
```

```

46
47 //bind
48 if (bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
49     error_handling("bind error");
50
51 //listen
52 if (listen(serv_sock, 2) == -1)
53     error_handling("listen error");
54
55 //accept
56 clnt_addr_size = sizeof(clnt_addr);
57 clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_addr, &clnt_addr_size);
58 if (clnt_sock == -1)
59     error_handling("accept error");
60
61 //데이터 주고받기
62
63 //1. client가 보낸 메시지 받기
64 if (read(clnt_sock, client_message, sizeof(client_message) - 1) == -1)
65     error_handling("read error");
66 printf("Message from client : %s\n", client_message);
67
68 //2. 파일 보내기
69
70 //파일 열기
71 fp = fopen("20180221_134217.mp4", "rb"); // 이진 읽기 모드
72 if (fp == NULL)
73     error_handling("file open error");
74
75 //후후 debug를 위해 파일 사이즈 전송
76 fseek(fp, 0, SEEK_END);
77 filesize = ftell(fp);
78 fseek(fp, 0, SEEK_SET);
79
80 write(clnt_sock, &filesize, sizeof(filesize));
81
82 //파일 전송
83 while (1)
84 {
85     read_cnt = fread((void*)buf, 1, BUFSIZE, fp);
86     if (read_cnt < BUFSIZE) {
87         write(clnt_sock, buf, read_cnt);
88         break;
89     }
90     write(clnt_sock, buf, BUFSIZE);
91 }
92
93 //3. 데이터 모두 전송 후, half-close(출력 스트림만 끊음)
94 if (shutdown(clnt_sock, SHUT_WR) == -1)
95     error_handling("shutdown error");

```

```

91 }
92
93 //3. 데이터 모두 전송 후, half-close(출력 스트림만 끊음)
94 if (shutdown(clnt_sock, SHUT_WR) == -1)
95     error_handling("shutdown error");
96
97 usleep(100);
98
99 //4. client가 보낸 메시지 받기
100 if (read(clnt_sock, client_debug, sizeof(client_debug) - 1) == -1)
101     error_handling("read error");
102 printf("Message from client : %s\n", client_debug);
103
104 //5. Good bye 메시지 보내기
105 write(clnt_sock, server_message, sizeof(server_message));
106
107 //파일 및 소켓 닫기
108 fclose(fp);
109 close(clnt_sock);
110 close(serv_sock);
111 return 0;
112 }
113
114 void error_handling(char* message)
115 {
116     fputs(message, stderr);
117     fputc('\n', stderr);
118     exit(1);
119 }

```

1-2. client 소스코드

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6  #include <sys/socket.h>
7
8  #define BUFSIZE 1460
9
10 void error_handling(char* message);
11
12 int main(int argc, char** argv)
13 {
14     int clnt_sock;
15     FILE* fp;
16
17     struct sockaddr_in serv_addr;
18
19     char buf[BUFSIZE];
20     char client_message[30] = "Hello Server"; // client가 server에 보내는 메시지
21     char debug_message[30] = "Same Size";
22     char debug_fail[30] = "Not Same Size";
23     char server_message[30]; // server가 client에 보낸 메시지를 저장
24
25     int read_cnt;
26     int filesize = 0;
27     int recvsz;
28
29
30     if (argc != 3) {
31         printf("Error : %s <IP> <port>\n", argv[0]);
32         exit(1);
33     }
34
35     //소켓 생성
36     clnt_sock = socket(PF_INET, SOCK_STREAM, 0);
37     if (clnt_sock == -1)
38         error_handling("socket() error");
39
40     //serv_addr 주소 지정
41     memset(&serv_addr, 0, sizeof(serv_addr));
42     serv_addr.sin_family = AF_INET;
43     serv_addr.sin_addr.s_addr = inet_addr(argv[1]);
44     serv_addr.sin_port = htons(atoi(argv[2]));
45
46     //connect
47     if (connect(clnt_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
48         error_handling("connect() error!");
49
50     //데이터 주고받기
```



```

49
50 //데이터 주고받기
51
52 //1. client가 "Hello Server"메시지 보내기
53 write(clnt_sock, client_message, sizeof(client_message));
54
55 //2. 파일 받기
56
57 //추후 debug를 위해 원본 파일 사이즈 받기
58 read(clnt_sock, &filesize, sizeof(filesize));
59 printf("original file: %d\n", filesize);
60
61 //파일 열기(파일이 존재하지 않으므로 빈 파일 새로 생성됨)
62 fp = fopen("receive.mp4", "wb"); // 이진 쓰기 모드
63 if (fp == NULL)
64     error_handling("file open error");
65
66 //파일 수신
67 while ((read_cnt = read(clnt_sock, buf, BUFSIZE)) != 0)
68 {
69     fwrite((void*)buf, 1, read_cnt, fp);
70 }
71
72 //3. 받은 파일과 원본 파일 사이즈 비교
73 fseek(fp, 0, SEEK_END);
74 recvsize = ftell(fp);
75 fseek(fp, 0, SEEK_SET);
76 printf("receive file: %d\n", recvsize);
77
78 usleep(100);
79
80 //4. 사이즈 비교 후, 알맞은 메시지 보내기
81 if (filesize == recvsize)
82     write(clnt_sock, debug_message, sizeof(debug_message));
83 else
84     write(clnt_sock, debug_fail, sizeof(debug_fail));
85
86 //4. Good bye 메시지 받기
87 if (read(clnt_sock, server_message, sizeof(server_message) - 1) == -1)
88     error_handling("read error!");
89 printf("Message from server: %s\n", server_message);
90
91 //파일 및 소켓 닫기
92 fclose(fp);
93 close(clnt_sock);
94 return 0;
95 }
96
97 void error_handling(char* message)
98 {

```

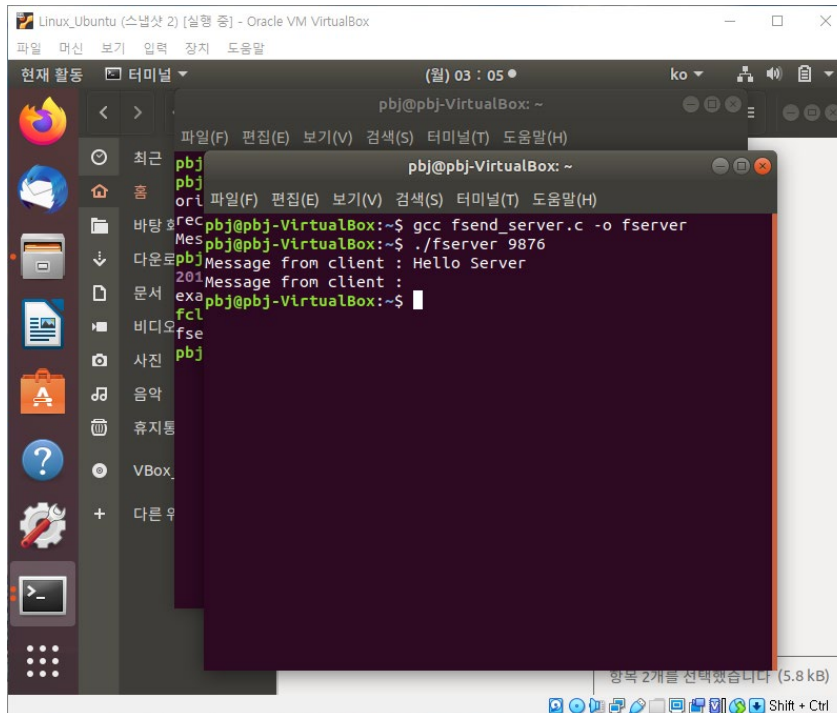
```

97 void error_handling(char* message)
98 {
99     fputs(message, stderr);
100     fputc('\n', stderr);
101     exit(1);
102 }

```

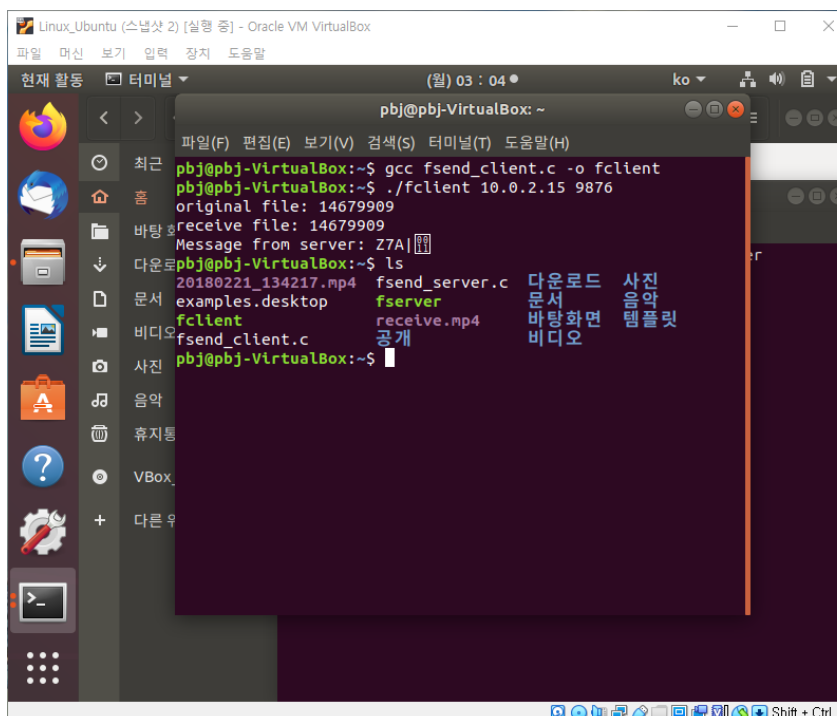
2. 실행 화면

2-1. server 쪽 실행 화면



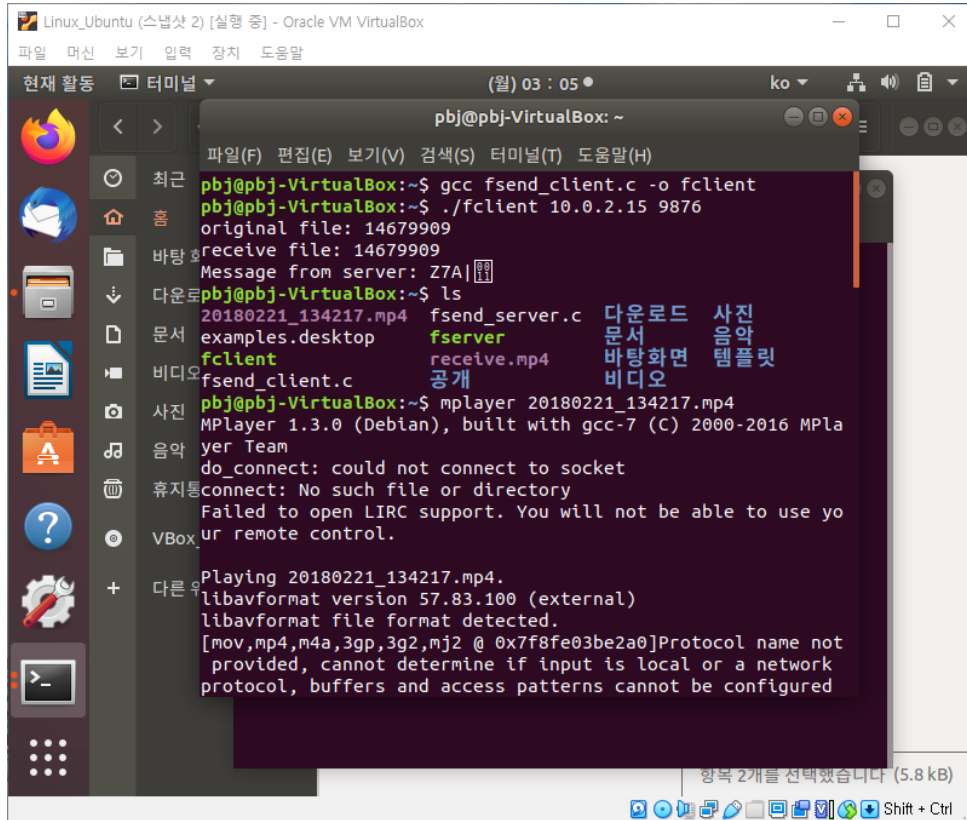
```
pbj@pbj-VirtualBox: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
pbj@pbj-VirtualBox: ~  
pbj  
ori  
rec pbj@pbj-VirtualBox:~$ gcc fsend_server.c -o fserver  
Mes pbj@pbj-VirtualBox:~$ ./fserver 9876  
pbj@pbj-VirtualBox:~$  
Message from client : Hello Server  
201  
Message from client :  
exa pbj@pbj-VirtualBox:~$  
fcl  
fse  
pbj
```

2-2. client 쪽 실행 화면

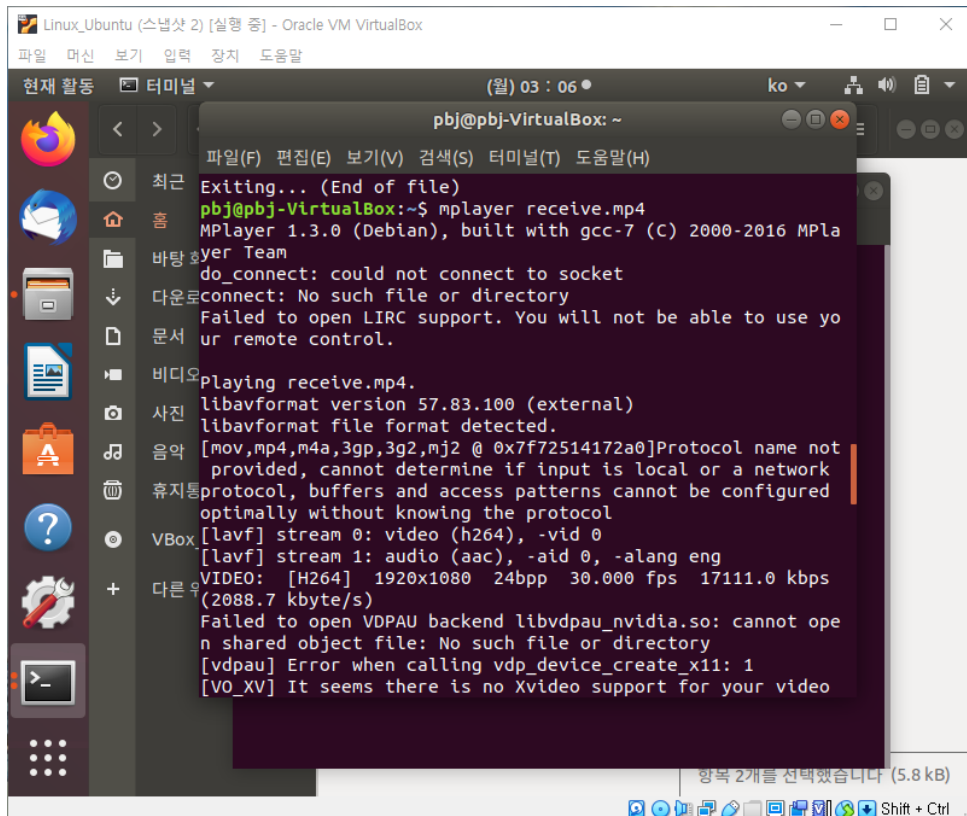


```
pbj@pbj-VirtualBox: ~  
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)  
pbj@pbj-VirtualBox:~$ gcc fsend_client.c -o fclient  
pbj@pbj-VirtualBox:~$ ./fclient 10.0.2.15 9876  
original file: 14679909  
receive file: 14679909  
Message from server: Z7A|  
pbj@pbj-VirtualBox:~$ ls  
20180221_134217.mp4 fserver 다운로드 사진  
examples.desktop fserver 문서 음악  
fclient receive.mp4 바탕화면 템플릿  
fsend_client.c 공개 비디오  
pbj@pbj-VirtualBox:~$
```

*실행 후, mplayer 를 통해 원본 영상 파일과 받은 영상 파일을 시청하며, 영상 전송이 잘 이루어졌는지 최종 확인합니다.



```
Linux_Ubuntu (스냅샷 2) [실행 중] - Oracle VM VirtualBox
파일  머신  보기  입력  장치  도움말
현재 활동  터미널
(월) 03 : 05
ko
pbj@pbj-VirtualBox: ~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
pbj@pbj-VirtualBox:~$ gcc fsend_client.c -o fclient
pbj@pbj-VirtualBox:~$ ./fclient 10.0.2.15 9876
original file: 14679909
receive file: 14679909
Message from server: Z7A
pbj@pbj-VirtualBox:~$ ls
20180221_134217.mp4  fsend_server.c  다운로드  사진
examples.desktop    fserver         문서      음악
fclient             receive.mp4     바탕화면  템플릿
fsend_client.c      공개           비디오
pbj@pbj-VirtualBox:~$ mplayer 20180221_134217.mp4
MPlayer 1.3.0 (Debian), built with gcc-7 (C) 2000-2016 MPla
yer Team
do_connect: could not connect to socket
connect: No such file or directory
Failed to open LIRC support. You will not be able to use yo
ur remote control.
Playing 20180221_134217.mp4.
libavformat version 57.83.100 (external)
libavformat file format detected.
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x7f8fe03be2a0]Protocol name not
provided, cannot determine if input is local or a network
protocol, buffers and access patterns cannot be configured
항목 2개를 선택했습니다 (5.8 kB)
```



```
Linux_Ubuntu (스냅샷 2) [실행 중] - Oracle VM VirtualBox
파일  머신  보기  입력  장치  도움말
현재 활동  터미널
(월) 03 : 06
ko
pbj@pbj-VirtualBox: ~
파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
Exiting... (End of file)
pbj@pbj-VirtualBox:~$ mplayer receive.mp4
MPlayer 1.3.0 (Debian), built with gcc-7 (C) 2000-2016 MPla
yer Team
do_connect: could not connect to socket
connect: No such file or directory
Failed to open LIRC support. You will not be able to use yo
ur remote control.
Playing receive.mp4.
libavformat version 57.83.100 (external)
libavformat file format detected.
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x7f72514172a0]Protocol name not
provided, cannot determine if input is local or a network
protocol, buffers and access patterns cannot be configured
optimally without knowing the protocol
[lavf] stream 0: video (h264), -vid 0
[lavf] stream 1: audio (aac), -aid 0, -alang eng
VIDEO: [H264] 1920x1080 24bpp 30.000 fps 17111.0 kbps
(2088.7 kbyte/s)
Failed to open VDPAU backend libvdpau_nvidia.so: cannot ope
n shared object file: No such file or directory
[vdpau] Error when calling vdp_device_create_x11: 1
[VO_XV] It seems there is no Xvideo support for your video
항목 2개를 선택했습니다 (5.8 kB)
```

3. 구현/개발 내용

소켓 생성, bind, listen, accept, close 등 TCP 통신의 기본적인 과정들은 Mission1 의 내용을 참고했습니다. 이후, <https://xxun.tistory.com/123> 의 소스코드에서 파일 열기, 파일 송신, 파일 수신 과정에 필요한 코드들을 참고하여 파일 송수신의 기본적인 틀을 잡았습니다. 또한, 파일 전송을 마친 후 server 쪽에서의 half-close 방식도 참고했습니다.

구체적으로, client 는 server 에 접속하여 "Hello Server" 메시지를 전송합니다. 메시지를 받은 server 는 client 가 보낸 메시지를 출력합니다. 이후, server 는 본격적으로 영상 파일을 보내기 위해 보내고자 하는 영상 파일을 열고, 파일을 전송합니다. 파일 전송이 모두 끝난 후, server 에서는 half-close 방식으로 출력 스트림만 끊습니다. 이때 client 에서는 원본 파일과 받은 파일의 사이즈를 비교하여, 같은 사이즈라면 "Same Size"의 메시지를, 다른 사이즈라면 "Different Size"의 메시지를 전송합니다. 이 메시지를 server 쪽에서 출력하도록 하고, client 와 server 모두 파일과 소켓을 닫습니다.

구현한 내용 중 특별히 신경을 쓴 부분은 다음과 같습니다. 우선 server 에서 미리 지정한 원본 파일을 열면서, 이 원본 파일의 사이즈를 client 에 전송했습니다. 이를 통해 원본 파일의 사이즈를 받은 client 는, 영상 파일을 모두 받은 후, 원본 파일의 사이즈와 받은 파일의 사이즈가 같은 지 비교할 수 있게 됩니다. 그리고 이 비교를 바탕으로, 각 상황에 알맞은 메시지를 client 가 전송하도록 했습니다. 다음으로, 영상 파일 전송 시 BUFSIZE 를 1460 으로 설정하여, 1460 바이트의 패킷으로 나누어 계속 전달할 수 있도록 했습니다. 그리고 계속 1460 바이트로 보내다가 전송할 데이터의 사이즈가 1460 바이트 미만이면, 전송할 데이터의 마지막이라고 판단하여 이 데이터의 전송을 마지막으로 파일 전송을 종료하도록 했습니다. 마지막으로, 데이터를 모두 전송한 server 에서는 half-

close 로 출력 스트림을 닫고, client 가 보낸 사이즈 비교 메시지만을 받을 수 있도록 했습니다.